**Figure 13.7 A flowchart describing the workflow of k-means, implemented with iterative MapReduce.**

For these reasons, rather than running `m` times MapReduce separately, we can use a more efficient (in this context) evolution of this programming model, *Iterative MapReduce*[12].

The idea is that we spin up mappers and reducers once, sharding data during the configuration of the mappers and assigning points shards to each mapper only once; then we iterate the classic MapReduce cycle until needed, passing only the current list of centroids as input to *all* the mappers. Thus, the amount of data per job to be passed to each mapper is going to be several orders of magnitude smaller than the dataset size, and ideally it will also be significantly smaller than the size of each shard. We can pass the number of mappers to create as an argument to our enhanced k-means method, and tune this parameter based on the size of the dataset and the capacity of each mapper.

## Listing 13.2 MapReduce k-means

```
function MRkmeans(points, numShards, numClusters, maxIter, T1,
T2)   #1
  shards ← randomShard(points,
numShards)                              #2
  centroids ← MRcanopyCentroids(points, numClusters, T1,
T2)         #3
  mappers ← initMappers(numShards, classifyPoints,
shards)           #4
  reducers ← initReducers(numClusters,
centerOfMass)                 #5
```